Homework 1: Likelihood, Floating-Point Arithmetic, and Reproducibility

Course: Computational Statistics
Author: Kazi Habib Ullah

In [ ]:
```
%pip install numpy matplotlib
```

Question 1: When likelihood disappears

In [144…
```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(123)
n = 20_000
y = rng.binomial(1, 0.03, size=n)
print("Number of One's =", y.sum())
print("Sample mean=", y.mean())
```

```
Number of One's = 583
Sample mean= 0.02915
```

Sample mean 0.02915 is close to 0.03 as given.

Question 1.1: Likelihood and Log-likelihood function manually evaluating

In [145…
```
def likelihood(p, y):
    return np.prod((p**y) * ((1 - p)**(1 - y)))

def log_likelihood(p, y):
    return np.sum(y * np.log(p) + (1 - y) * np.log(1 - p))
```

In [146…
```
print("likelihood at p = 0.03 is", likelihood(0.03, y))
print("log_likelihood at p = 0.03 is" , log_likelihood(0.03, y))
```

```
likelihood at p = 0.03 is 0.0
log_likelihood at p = 0.03 is -2635.749685868136
```

Question 1.2: Function evaluation over grid

I am creating a grid of 400 equally spaced points from 0.001 to 0.1 and evaluating both likelihood and log_likelihood functions.

In [147…
```
p_grid = np.linspace(0.001, 0.10, 400)

Likelihood_vals = np.array([likelihood(p, y) for p in p_grid])
```

```
log_likelihood_vals = np.array([log_likelihood(p, y) for p in p_grid])


zero_count = np.sum(Likelihood_vals == 0.0)

print(f"Total grid points: {len(p_grid)}")
print(f"Number of grid points where likelihood exactly zero: {zero_count}")
```

```
Total grid points: 400
Number of grid points where likelihood exactly zero: 400
```

Explanation:
Likelihood is 0 for every 400 values of p because after multipying 20000 small numbers (0.001 to 0.1) the total likelihood becomes extermely close to 0.0 that our 64 bit computer cannot preserve the number. So it becomes 0 for all grid values.

Question 1.3: Log-Likelihood Plotting

Now I will find the grid that maximizes $l(p)$ and compare it with the Maximum Likelihood Estimator (MLE).

In [152…

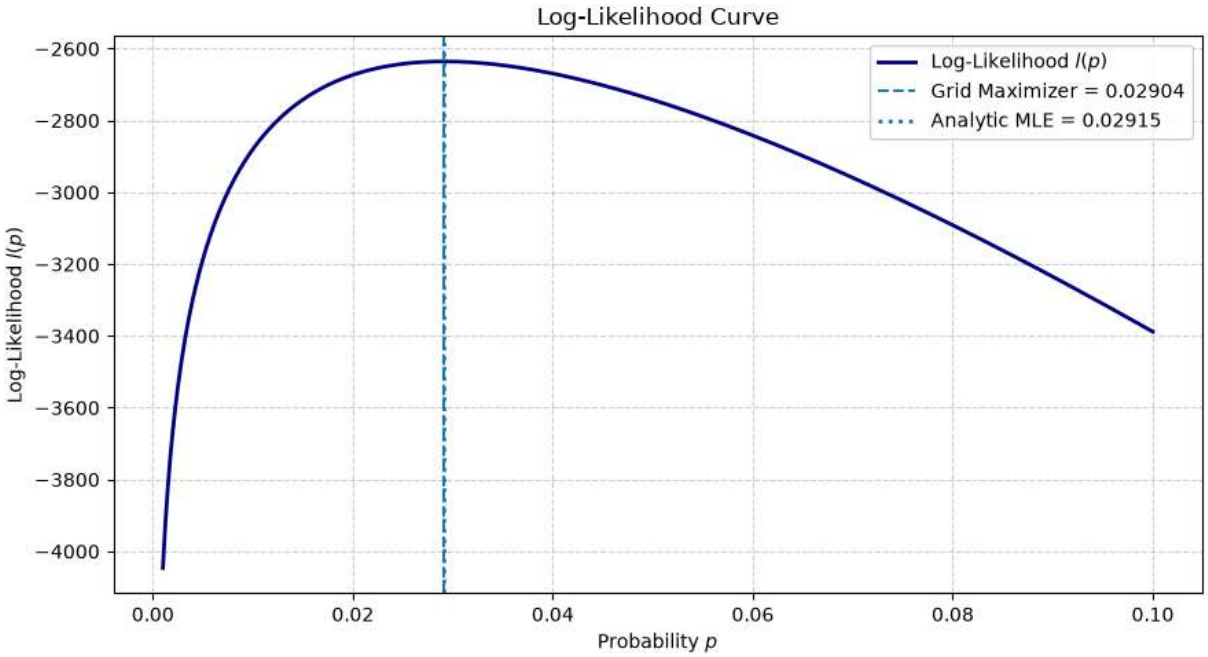
```
# Identifying grid maximizer and computing analytic MLE
grid_mle_idx = np.argmax(log_likelihood_vals)
grid_mle = p_grid[grid_mle_idx]
analytic_mle = np.mean(y)

print("Grid Maximizer of log-likelihood:", grid_mle)
print("Analytic MLE :", analytic_mle)
print("Absolute Difference:", abs(grid_mle - analytic_mle))

# Ploting the log-likelihood curve
plt.figure(figsize=(9, 5))
plt.plot(p_grid, log_likelihood_vals, label="Log-Likelihood $l(p)$", color="navy",
plt.axvline(grid_mle, linestyle="--", linewidth=1.5,
            label=f"Grid Maximizer = {grid_mle:.5f}")
plt.axvline(analytic_mle, linestyle=":", linewidth=2,
            label=f"Analytic MLE = {analytic_mle:.5f}")

plt.xlabel("Probability $p$")
plt.ylabel("Log-Likelihood $l(p)$")
plt.title("Log-Likelihood Curve")
plt.legend(frameon=True, facecolor="white", framealpha=0.9)
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()
```

```
Grid Maximizer of log-likelihood: 0.02903759398496241
Analytic MLE : 0.02915
Absolute Difference: 0.00011240601503758815
```



Explanation:
The grid maximizer (≈ 0.02904) and the analytic MLE (0.02915) do not match because when we created the grid of 400 equally spaced values it did not took the value of analytic MLE exactly.

Question 1.4: Statistical Equivalence and Computational Safety

We now that the natural logarithm function $\ln(x)$ is a strictly monotonically increasing function, any value p that maximizes the likelihood $L(p)$ also maximizes the log-likelihood $l(p)$. Computationally, taking the logarithm converts products into sums, which converts calculation of multiplication of extremely small numbers to addition of them. Which ultimately eliminates the floating-point underflow.

Question 2

Question 2.1: Variance Calculation and Results Table

In [153…

```
# Generate dataset
rng = np.random.default_rng(123)
x64 = rng.normal(loc=1e8, scale=1.0, size=100_000)
x32 = x64.astype(np.float32)

# Defining the two variance formulas
def v_naive(x):
    return np.mean(x**2) - (np.mean(x))**2

def v_centered(x):
    return np.mean((x - np.mean(x))**2)
```

```
# Compute estimated variances
res_naive_64 = v_naive(x64)
res_naive_32 = v_naive(x32)

res_cent_64 = v_centered(x64)
res_cent_32 = v_centered(x32)

res_ref_64 = np.var(x64)
res_ref_32 = np.var(x32)

# table
print(f"{'Method':<20} | {'float64':<18} | {'float32':<18}")
print("-" * 60)
print(f"{'Naive':<20} | {res_naive_64:<18.6f} | {res_naive_32:<18.6f}")
print(f"{'Centered':<20} | {res_cent_64:<18.6f} | {res_cent_32:<18.6f}")
print(f"{'np.var (Reference)':<20} | {res_ref_64:<18.6f} | {res_ref_32:<18.6f}")
```

```
Method               | float64            | float32
------------------------------------------------------------
Naive                | 0.000000           | 2147483648.000000
Centered             | 0.999621           | 256.010895
np.var (Reference)   | 0.999621           | 256.010895
```

Question 2.2: Absolute Error

In the following table I will compute the absolute error $|v - 1|$ against the true generating variance $\sigma^2 = 1.0$.

In [162…

```
true_var = 1.0

# Absolute errors
err_naive_64 = abs(res_naive_64 - true_var)
err_naive_32 = abs(res_naive_32 - true_var)

err_cent_64 = abs(res_cent_64 - true_var)
err_cent_32 = abs(res_cent_32 - true_var)

err_ref_64 = abs(res_ref_64 - true_var)
err_ref_32 = abs(res_ref_32 - true_var)

# Table
print(f"{'Method':<20} | {'float64 Error':<18} | {'float32 Error':<18}")
print("-" * 60)
print(f"{'Naive':<20} | {err_naive_64:<18.6f} | {err_naive_32:<18.6f}")
print(f"{'Centered':<20} | {err_cent_64:<18.6f} | {err_cent_32:<18.6f}")
print(f"{'np.var (Reference)':<20} | {err_ref_64:<18.6f} | {err_ref_32:<18.6f}")
```

```
Method               | float64 Error      | float32 Error
------------------------------------------------------------
Naive                | 1.000000           | 2147483648.000000
Centered             | 0.000379           | 255.010895
np.var (Reference)   | 0.000379           | 255.010895
```

Inaccuracy of Naive Formula:

The naive formula $v_{\text{naive}} = \text{mean}(x^2) - \{\text{mean}(x)\}^2$ undergoes through severe

cancellation. Because we know the order of mean μ is 10^8, so both $\text{mean}(x^2)$ and $\{\text{mean}(x)\}^2$ becomes 10^{16} when calculated. Although the true variance is only 1.0. Subtracting two huge, nearly equal floating-point numbers neglects the lower-order digits that contain the variance information. Which ultimate produce large errors in float64 and complete failure values in float32.

Numerical and Statistical Error:

- **Numerical Errors:** The differences between float32 and float64 results and the deviation of the naive formula from the centered formula, are purely numerical error caused by using finite-precision floating-point arithmetic and roundoff error.
- **Statistical Error:** When using float64, we got small difference for the central and refference formula (0.999621) with the true variance value $\sigma^2 = 1.0$. This is a statistical error due to using a finite random sample of $N = 100,000$ observations.

In [163…
```
import numpy as np
import matplotlib as mpl

print(f"NumPy version: {np.__version__}")
print(f"Matplotlib version: {mpl.__version__}")
```

```
NumPy version: 2.5.3
Matplotlib version: 3.11.1
```